

```
In [1]: ##installa dipendenze sulla destra
```

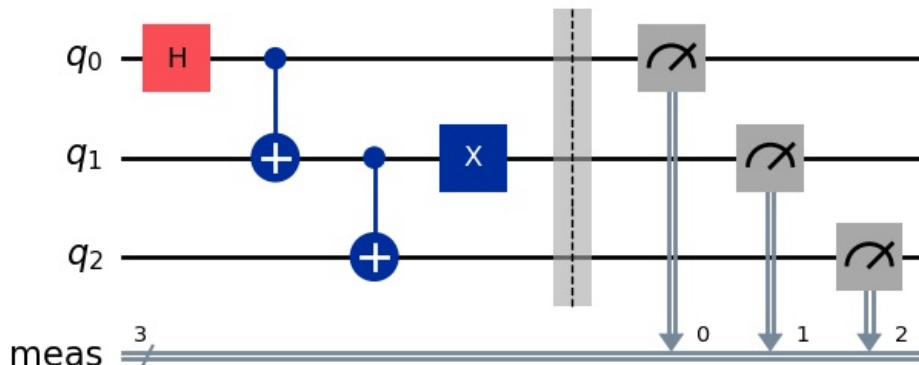
```
In [2]: from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler
```

```
In [3]: qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.x(1)
qc.measure_all()
```

```
In [4]: sampler = StatevectorSampler()
result = sampler.run([qc], shots=pow(2,10)).result()
print(result[0].data.meas.get_counts())
qc.draw("mpl")

{'010': 495, '101': 529}
```

Out[4]:



```
In [5]: import matplotlib.pyplot as plt
from qiskit.visualization import plot_histogram
```

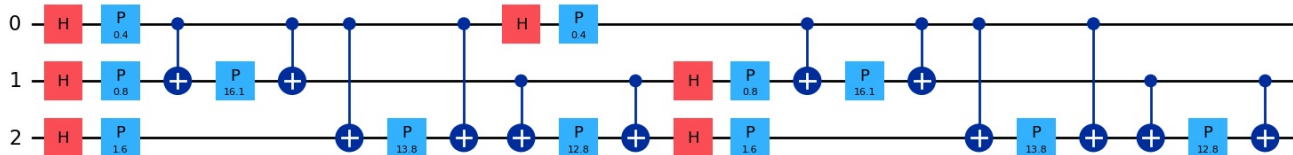
```
In [6]: counts = result[0].data.meas.get_counts()
plot_histogram(counts)
plt.show()
```

```
In [7]: from qiskit.circuit.library import zz_feature_map
```

```
In [8]: features = [0.2, 0.4, 0.8]
feature_map = zz_feature_map(feature_dimension=len(features))
```

```
In [9]: encoded = feature_map.assign_parameters(features)
encoded.draw("mpl")
```

Out[9]:



```
In [10]: import matplotlib.pyplot as plt
from qiskit import QuantumCircuit
from qiskit_ibm_runtime.fake_provider import FakeAuckland, FakeWashingtonV2
from qiskit.transpiler import generate_preset_pass_manager
```

ModuleNotFoundError Traceback (most recent call last)

Cell In[10], line 3

```
1 import matplotlib.pyplot as plt
2 from qiskit import QuantumCircuit
----> 3 from qiskit_ibm_runtime.fake_provider import FakeAuckland, FakeWashingtonV2
4 from qiskit.transpiler import generate_preset_pass_manager
```

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/_init_.py:204

```
201 import logging
202 import warnings
--> 204 from .qiskit_runtime_service import QiskitRuntimeService
205 from .ibm_backend import IBMBackend
206 from .runtime_job_v2 import RuntimeJobV2
```

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/qiskit_runtime_service.py:26

```

23 from qiskit.providers.exceptions import QiskitBackendNotFoundError
24 from qiskit.providers.providerutils import filter_backends
--> 26 from qiskit_ibm_runtime import ibm_backend
27 from .proxies import ProxyConfiguration
28 from .utils import is_crn

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/ibm_backend.py:35
26 from .models import (
27     BackendStatus,
28     BackendProperties,
29     GateConfig,
30     QasmBackendConfiguration,
31 )
33 from . import qiskit_runtime_service # pylint: disable=unused-import,cyclic-import
--> 35 from .api.clients import RuntimeClient
36 from .exceptions import (
37     IBMBackendApiProtocolError,
38     IBMBackendError,
39 )
40 from .utils.backend_converter import convert_to_target

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/api/clients/__init__.py:16
13 """IBM Quantum API clients."""
15 from .version import VersionClient
--> 16 from .runtime import RuntimeClient

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/api/clients/runtime.py:24
22 from .backend import BaseBackendClient
23 from ..rest.runtime import Runtime
--> 24 from ..client_parameters import ClientParameters
26 logger = logging.getLogger(__name__)
29 class RuntimeClient(BaseBackendClient):

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/api/client_parameters.py:17
15 from typing import Any
16 from collections.abc import Callable
--> 17 from ..proxies import ProxyConfiguration
19 from ..utils import default_runtime_url_resolver
20 from ..api.auth import CloudAuth

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/proxies/__init__.py:17
1 # This code is part of Qiskit.
2 #
3 # (C) Copyright IBM 2022.
(...)
10 # copyright notice, and modified files need to carry a notice indicating
11 # that they have been altered from the originals.
13 """
14 Proxy configuration.
15 """
--> 17 from .configuration import ProxyConfiguration

File /opt/anaconda3/lib/python3.12/site-packages/qiskit_ibm_runtime/proxies/configuration.py:20
17 from typing import Any
18 from urllib.parse import urlparse
--> 20 from requests_ntlm import HttpNtlmAuth
23 @dataclass
24 class ProxyConfiguration:
25     """Class for representing a proxy configuration.
26
27     Args
28     (...)
31         password_ntlm: password used to enable NTLM user authentication.
32         """

File /opt/anaconda3/lib/python3.12/site-packages/requests_ntlm/__init__.py:1
--> 1 from .requests_ntlm import HttpNtlmAuth
3 __all__ = ("HttpNtlmAuth",)

File /opt/anaconda3/lib/python3.12/site-packages/requests_ntlm/requests_ntlm.py:9
6 from urllib.parse import urlparse
8 import requests
--> 9 import spnego
10 from cryptography import x509
11 from cryptography.exceptions import UnsupportedAlgorithm

File /opt/anaconda3/lib/python3.12/site-packages/spnego/__init__.py:9
6 import logging.config
7 import os
--> 9 from spnego._context import (
10     ContextProxy,
11     ContextReq,
12     IOVUnwrapResult,
13     IOVWrapResult,
14     UnwrapResult,
15     WinRMWrapResult,
16     WrapResult,
17 )
18 from spnego._credential import (

```

```

19     Credential,
20     CredentialCache,
(...)
24     Password,
25 )
26 from spnego.auth import client, server

File /opt/anaconda3/lib/python3.12/site-packages/spnego/_context.py:10
7 import typing
8 import warnings
---> 10 from spnego._credential import Credential
11 from spnego._text import to_text
12 from spnego.channel_bindings import GssChannelBindings

File /opt/anaconda3/lib/python3.12/site-packages/spnego/_credential.py:14
11 import dataclasses
12 import typing
---> 14 from spnego._ntlm_raw.crypto import is_ntlm_hash
15 from spnego.exceptions import InvalidCredentialError, NoCredentialError
17 # FUTURE: Add Anonymous credential class

File /opt/anaconda3/lib/python3.12/site-packages/spnego/_ntlm_raw/crypto.py:23
20 import typing
22 from cryptography.hazmat.backends import default_backend
---> 23 from cryptography.hazmat.primitives.ciphers import Cipher
25 from spnego._ntlm_raw.des import DES
26 from spnego._ntlm_raw.md4 import md4

File /opt/anaconda3/lib/python3.12/site-packages/cryptography/hazmat/primitives/ciphers/__init__.py:11
5 from __future__ import annotations
7 from cryptography.hazmat.primitives._cipheralgorithm import (
8     BlockCipherAlgorithm,
9     CipherAlgorithm,
10 )
---> 11 from cryptography.hazmat.primitives.ciphers.base import (
12     AEADCipherContext,
13     AEADDecryptionContext,
14     AEADEncryptionContext,
15     Cipher,
16     CipherContext,
17 )
19 __all__ = [
20     "AEADCipherContext",
21     "AEADDecryptionContext",
(...)
26     "CipherContext",
27 ]

File /opt/anaconda3/lib/python3.12/site-packages/cryptography/hazmat/primitives/ciphers/base.py:10
7 import abc
8 import typing
---> 10 from cryptography.hazmat.bindings._rust import openssl as rust_openssl
11 from cryptography.hazmat.primitives._cipheralgorithm import CipherAlgorithm
12 from cryptography.hazmat.primitives.ciphers import modes

```

ModuleNotFoundError: No module named '_cffi_backend'

```

In [11]: backend = FakeAuckland()
ghz = QuantumCircuit(15)
ghz.h(0)
ghz.cx(0, range(1, 15))
depths = []
for seed in range(100):
    pass_manager = generate_preset_pass_manager(
        optimization_level=1,
        backend=backend,
        layout_method="trivial", # Fixed layout mapped in circuit order
        seed_transpiler=seed, # For reproducible results
    )
    depths.append(pass_manager.run(ghz).depth())

pass_manager.run(ghz).draw("mpl")

# plt.figure(figsize=(8, 6))
# plt.hist(depths, align="left", color="#AC557C")
# plt.xlabel("Depth", fontsize=14)
# plt.ylabel("Counts", fontsize=14)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 backend = FakeAuckland()
      2 ghz = QuantumCircuit(15)
      3 ghz.h(0)

NameError: name 'FakeAuckland' is not defined

```

```

In [12]: from qiskit import QuantumCircuit
from qiskit.circuit import QuantumCircuit, Parameter

```

```
In [13]: circuit = QuantumCircuit(2)
circuit.h(0)
circuit.cx(0,1)
circuit.ry(0,0,"a")
circuit.measure_all()
```

```
In [14]: from qiskit.circuit import QuantumCircuit, Parameter
from qiskit_aer.primitives import SamplerV2
```

```
In [15]: # create the parameter
phi = Parameter('phi')
params = [0.1] * circuit.num_parameters
pub = (circuit, params, 1024)
sampler = SamplerV2()
job = sampler.run([pub])
result = job.result()
```

```
In [16]: data = result[0].data
print(f"Databin: {data}\n")
# to access the BitArray, use the key "meas", which is the default name of
# the classical register when this is added by the `measure_all` method
array = data.meas
print(f"BitArray: {array}\n")
print(f"The shape of register `meas` is {data.meas.array.shape}.\n")
print(f"The bytes in register `alpha`, shot by shot:\n{data.meas.array}\n")
# optionally, convert away from the native BitArray format to a dictionary format
counts = data.meas.get_counts()
print(f"Counts: {counts}")
```

Databin: DataBin(meas=BitArray(<shape=(), num_shots=1024, num_bits=2>))

BitArray: BitArray(<shape=(), num_shots=1024, num_bits=2>)

The shape of register `meas` is (1024, 1).

The bytes in register `alpha`, shot by shot:

```
[[0]
 [3]
 [0]
 ...
 [3]
 [3]
 [3]]
```

Counts: {'00': 521, '11': 503}

```
In [17]: from qiskit.circuit.library import efficient_su2
from qiskit.quantum_info import SparsePauliOp
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import EstimatorV2 as Estimator
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit_aer.primitives import SamplerV2
```

```
In [18]: n_qubits = 8
circuit = efficient_su2(n_qubits)
circuit.measure_all()
circuit.draw("mpl")
observable = SparsePauliOp("Z" * n_qubits)
params = [0.1] * circuit.num_parameters

exact_estimator = SamplerV2()
# The circuit needs to be transpiled to the AerSimulator target
pass_manager = generate_preset_pass_manager(3, AerSimulator())
isa_circuit = pass_manager.run(circuit)
pub = (isa_circuit, params)
job = exact_estimator.run([pub], shots=1024)
result = job.result()
counts_ideal = result[0].data.meas.get_counts()
counts_ideal
# to access the BitArray, use the key "meas", which is the default name of
# the classical register when this is added by the `measure_all` method
```

```
Out[18]: {'00000000': 915,  
          '10000000': 42,  
          '11000000': 7,  
          '00111100': 5,  
          '00000100': 2,  
          '01000000': 7,  
          '00001010': 4,  
          '00010000': 2,  
          '00011000': 3,  
          '01010000': 4,  
          '11100000': 4,  
          '00000010': 2,  
          '01100000': 2,  
          '01111000': 1,  
          '00101000': 2,  
          '00001111': 1,  
          '00100000': 2,  
          '00000011': 3,  
          '00110000': 4,  
          '11110000': 2,  
          '00000101': 2,  
          '10100000': 6,  
          '00111101': 1,  
          '10001100': 1}
```

```
In [19]: noise_model = NoiseModel()  
cx_depolarizing_prob = 0.02  
noise_model.add_all_qubit_quantum_error(  
    depolarizing_error(cx_depolarizing_prob, 2), ["cx"])  
  
noisy_estimator = SamplerV2(  
    options=dict(backend_options=dict(noise_model=noise_model)))  
job = noisy_estimator.run([pub])  
result = job.result()  
counts_ideal = result[0].data.meas.get_counts()  
counts_ideal
```

```
Out[19]: {'00000000': 674,
'00000011': 11,
'11000000': 17,
'00011000': 16,
'00101000': 21,
'10100000': 9,
'01000000': 23,
'01111000': 4,
'11100010': 2,
'01111110': 1,
'11111010': 1,
'00010100': 12,
'01100000': 10,
'11100000': 8,
'00110000': 16,
'10000000': 31,
'10010000': 1,
'00001100': 16,
'11101000': 1,
'11101010': 1,
'11101110': 1,
'00011011': 1,
'00000001': 8,
'00001000': 9,
'00000101': 11,
'01001000': 1,
'00110101': 1,
'11110000': 10,
'00001010': 7,
'00000010': 7,
'01100001': 1,
'00010000': 8,
'00001111': 3,
'11111111': 1,
'00001101': 1,
'01010000': 4,
'00111100': 4,
'10011000': 2,
'00000110': 11,
'00011110': 8,
'10000100': 1,
'00100110': 1,
'10010100': 2,
'00101010': 1,
'11000001': 1,
'00100000': 9,
'00111000': 2,
'01000100': 1,
'11111000': 2,
'11001000': 1,
'00000100': 5,
'00001001': 3,
'00100100': 1,
'10001000': 3,
'11110100': 1,
'00011100': 1,
'00111110': 1,
'11010000': 2,
'11000100': 1,
'01100110': 1,
'10011110': 1,
'00010010': 1,
'01100010': 1,
'11110110': 1,
'10101000': 1,
'00010111': 1,
'01000011': 1,
'11011000': 1,
'10110110': 1,
'11010100': 1}
```

```
In [20]: import pennylane as qml
from jax import numpy as np
import jax
```

/opt/anaconda3/lib/python3.12/site-packages/pennylane/capture/capture_operators.py:33: RuntimeWarning: PennyLane is not yet compatible with JAX versions > 0.4.28. You have version 0.9.0.1 installed. Please downgrade JAX to <=0.4.28 to avoid runtime errors.
warnings.warn(

AttributeError Traceback (most recent call last)

Cell In[20], line 1
----> 1 import pennylane as qml
2 from jax import numpy as np
3 import jax

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/__init__.py:21
14 """

```

15 This is the top level module from which all basic functions and classes of
16 PennyLane can be directly imported.
17 """
20 from pennylane.boolean_fn import BooleanFn
---> 21 import pennylane.numpy
22 from pennylane.queueing import QueuingManager, apply
24 import pennylane.compiler

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/numpy/__init__.py:87
84 from autograd import numpy as _np
85 from autograd.numpy import *
---> 87 from .wrapper import extract_tensors, tensor_wrapper, wrap_arrays
89 wrap_arrays(_np.__dict__, globals())
91 # Delete the unwrapped fft, linalg, random modules
92 # so that we can re-import our wrapped versions.

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/numpy/wrapper.py:23
19 from collections.abc import Sequence
21 from autograd import numpy as _np
---> 23 from .tensor import tensor
26 def extract_tensors(x):
27     """Iterate through an iterable, and extract any PennyLane
28     tensors that appear.
29
30     (...)
43     [tensor(0.1, requires_grad=True)]
44     """

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/numpy/tensor.py:25
22 from autograd.numpy.numpy_vspaces import ArrayVSpace, ComplexArrayVSpace
23 from autograd.tracer import Box
---> 25 from pennylane.operation import Operator
27 __doc__ = "NumPy with automatic differentiation support, provided by Autograd and PennyLane."
30 # Hotfix since _np.asarray doesn't have a gradient rule defined.

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/operation.py:1760
1752     return cls(*data, wires=metadata[0], **hyperparameters_dict)
1755 # =====
1756 # Base Operation class
1757 # =====
-> 1760 class Operation(Operator):
1761     r"""Base class representing quantum gates or channels applied to quantum states.
1762
1763     Operations define some additional properties, that are used for external
1764     (...)
1765     can be useful for some applications where the instance has to be identified
1766     """
1767     @property
1768     def grad_method(self) -> Literal["A", "F", None]:

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/operation.py:106, in ABCMeta.__new__(mcls, name, bases, namespace, **kwargs)
105 def __new__(mcls, name, bases, namespace, /, **kwargs):
-> 106     cls = super().__new__(mcls, name, bases, namespace, **kwargs)
107     _abc_init(cls)
108     return cls

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/operation.py:684, in Operator.__init_subclass__(cls, **_)
682 def __init_subclass__(cls, **_):
683     register_pytree(cls, cls.flatten, cls.unflatten)
-> 684     cls._primitive = create_operator_primitive(cls)

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/capture/capture_operators.py:119, in create_operator_primitive(operator_type)
116 if not has_jax:
117     return None
-> 119 from .custom_primitives import NonInterpPrimitive # pylint: disable=import-outside-toplevel
121 primitive = NonInterpPrimitive(operator_type.__name__)
122 primitive.prim_type = "operator"

File /opt/anaconda3/lib/python3.12/site-packages/pennylane/capture/custom_primitives.py:34
30 TRANSFORM = "transform"
33 # pylint: disable=too-few-public-methods,abstract-method
---> 34 class QmlPrimitive(jax.core.Primitive):
35     """A subclass for JAX's Primitive that differentiates between different
36     classes of primitives."""
38     _prim_type: PrimitiveType = PrimitiveType.DEFAULT

File /opt/anaconda3/lib/python3.12/site-packages/jax/_src/deprecations.py:57, in deprecation_getattr.<locals>.getattr(name)
55 warnings.warn(message, DeprecationWarning, stacklevel=2)
56 return fn
-> 57 raise AttributeError(f"module {module!r} has no attribute {name!r}")

AttributeError: module 'jax.core' has no attribute 'Primitive'

```

In [21]: dev1 = qml.device("default.qubit", wires=4)

```
-----
NameError                                Traceback (most recent call last)
Cell In[21], line 1
----> 1 dev1 = qml.device("default.qubit", wires=4)

NameError: name 'qml' is not defined
```

```
In [22]: def circuit(params):
        qml.RX(params[0], wires=0)
        qml.RY(params[1], wires=0)
        return qml.expval(qml.PauliZ(0))
        params = np.array([0.54, 0.12])
        print(circuit(params))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[22], line 5
      3     qml.RY(params[1], wires=0)
      4     return qml.expval(qml.PauliZ(0))
----> 5 params = np.array([0.54, 0.12])
      6 print(circuit(params))

NameError: name 'np' is not defined
```